Name: Phan Tran Thanh Huy

ID: ITCSIU22056

Lab 9

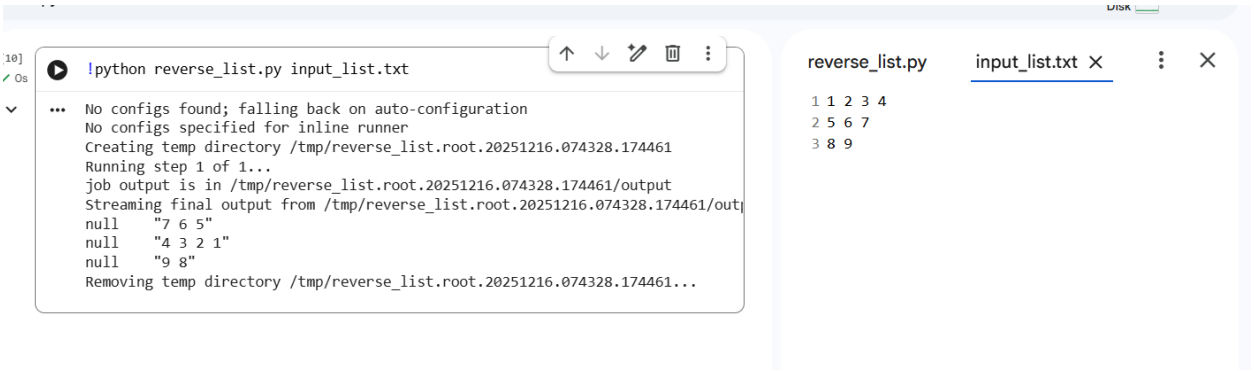
Exercise 1: Reverse a Linked List (Easy)

reverse_list.py

reverse_list.py

```python
from mrjob.job import MRJob

class MRReverseLinkedList(MRJob):

    def mapper(self, _, line):
        nodes = line.strip().split()

        reversed_nodes = nodes[::-1]

        reversed_line = " ".join(reversed_node

        yield None, reversed_line

if __name__ == '__main__':
    MRReverseLinkedList.run()
```

Result



Exercise 2: Detect Duplicate Elements in Arrays (Easy-Medium)
check_duplicates.py

```
%%file check_duplicates.py
from mrjob.job import MRJob

class MRDetectDuplicates(MRJob):

    def mapper_init(self):
        self.line_num = 0

    def mapper(self, _, line):
        self.line_num += 1

        elements = line.strip().split()

        has_duplicates = len(elements) != len(set(elements))

        yield self.line_num, has_duplicates

if __name__ == '__main__':
    MRDetectDuplicates.run()
```

```
Writing check_duplicates.py
```

Result

```
!python check_duplicates.py input.txt
```

```
No configs found; falling back on auto-configuration
No configs specified for inline runner
Creating temp directory /tmp/check_duplicates.root.20251216.074936.890373
Running step 1 of 1...
job output is in /tmp/check_duplicates.root.20251216.074936.890373/output
Streaming final output from /tmp/check_duplicates.root.20251216.074936.890373/
1       false
1       true
1       true
Removing temp directory /tmp/check_duplicates.root.20251216.074936.890373...
```

input.txt

```
1 1 2 2 3
2 4 5 6
3 7 7 7
```

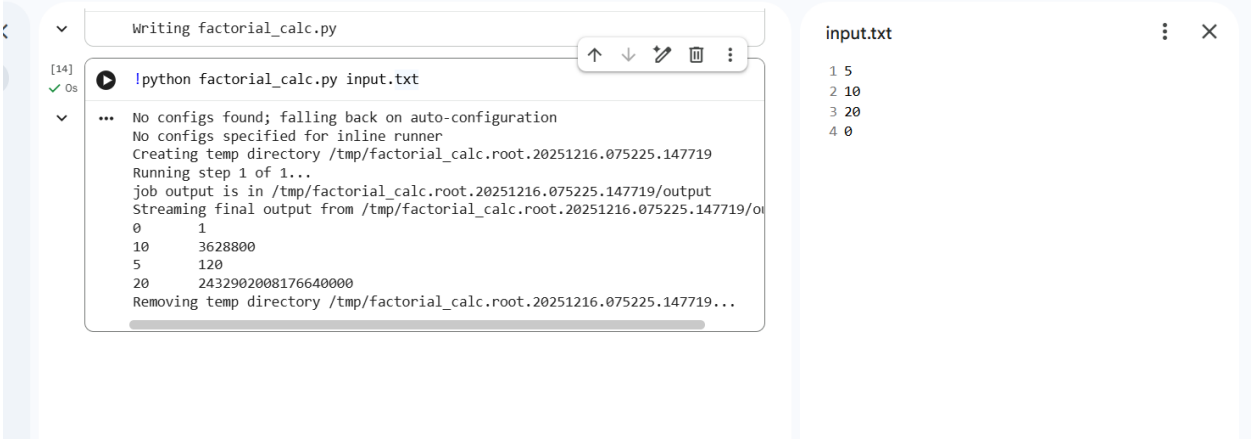
Exercise 3: Compute Factorial for Large Numbers (Medium)

factorial_calc.py

```
%%file factorial_calc.py
from mrjob.job import MRJob
import math

class MRFactorial(MRJob):

    def mapper(self, _, line):
        try:
            n = int(line.strip())

            result = math.factorial(n)

            yield n, result

        except ValueError:
            pass

if __name__ == '__main__':
    MRFactorial.run()
```

```
Writing factorial_calc.py
```

Result



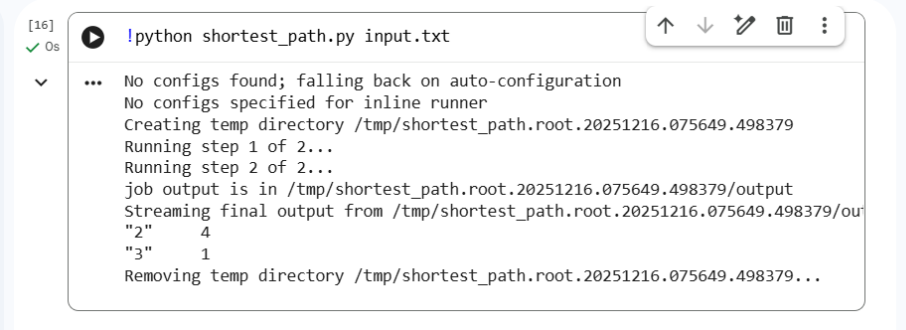
Exercise 4: Find Shortest Paths in a Graph (Medium-Hard)

shortest_path.py

```
%%file shortest_path.py
from mrjob.job import MRJob
from mrjob.step import MRStep

class MRShortestPath(MRJob):

    def configure_args(self):
        super(MRShortestPath, self).configure_args()
        self.add_passthru_arg('--source', default='1', help='Source Node ID'

    def steps(self):
        return [
            MRStep(mapper=self.mapper_build_graph,
                   reducer=self.reducer_build_graph),
            MRStep(mapper=self.mapper_propagate,
                   reducer=self.reducer_find_min)
        ]

    def mapper_build_graph(self, _, line):
        src, dest, weight = line.strip().split()

        yield src, ('EDGE', dest, int(weight))

        yield dest, ('NODE', )

    def reducer_build_graph(self, node, values):
        edges = {}
        distance = 0 if node == self.options.source else 999999
```

```
            for val in values:
                if val[0] == 'EDGE':
                    edges[val[1]] = val[2]

            yield node, (distance, edges)

        def mapper_propagate(self, node, node_data):
            distance, edges = node_data

            yield node, ('STRUCTURE', distance, edges)

            if distance < 999999:
                for neighbor, weight in edges.items():
                    new_dist = distance + weight
                    yield neighbor, ('DISTANCE', new_dist)

        def reducer_find_min(self, node, values):
            edges = {}
            min_dist = 999999

            for val in values:
                if val[0] == 'STRUCTURE':
                    current_dist = val[1]
                    edges = val[2]

                    if current_dist < min_dist:
                        min_dist = current_dist
                elif val[0] == 'DISTANCE':
                    if val[1] < min_dist:
                        min_dist = val[1]

                    min_dist = val[1]

            if min_dist < 999999 and node != self.options.source:
                yield node, min_dist

if __name__ == '__main__':
    MRShortestPath.run()
```

Result



Exercise 5: Find Shortest Paths in a Graph (Medium-Hard)

exercise5.py

```
%%file exercise_5.py
from mrjob.job import MRJob
from mrjob.step import MRStep
import random
import os
import math

class MRRSAKeyGen(MRJob):

    def steps(self):
        return [
            MRStep(mapper_init=self.mapper_init_prime,
                   mapper=self.mapper_find_primes,
                   reducer=self.reducer_select_pair),
            MRStep(mapper=self.mapper_compute_rsa,
                   reducer=self.reducer_output_rsa)
        ]

    def mapper_init_prime(self):
        random.seed(os.getpid())
        self.bit_length = 128
        self.k = 40

    def mapper_find_primes(self, _, line):
        num_candidates = int(line.strip())
        primes = []

        for _ in range(num_candidates):
            candidate = random.getrandbits(self.bit_length) | 1
            if self.miller_rabin(candidate, self.k):
```

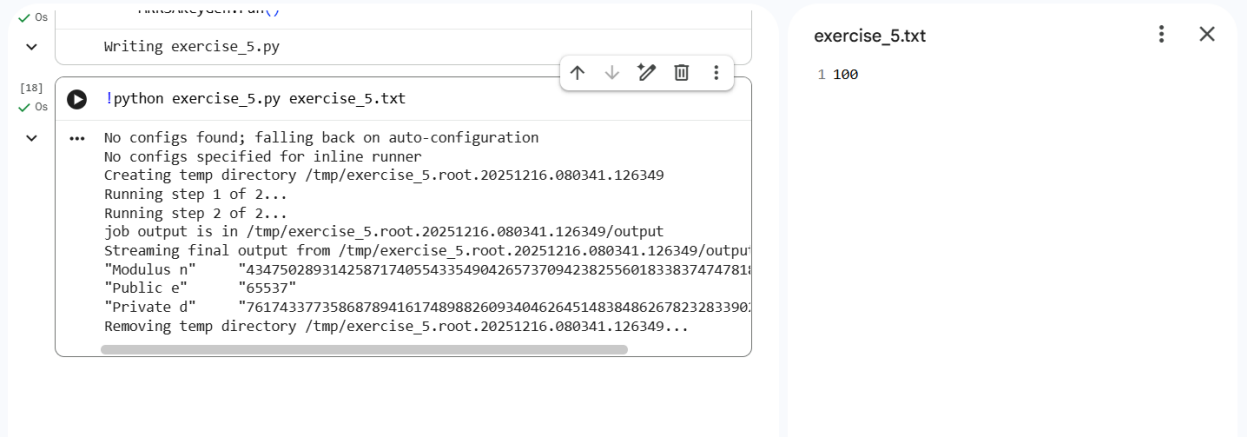
```
                primes.append(candidate)

        if len(primes) >= 2:
            yield None, (primes[0], primes[1])

    def reducer_select_pair(self, _, pairs):
        for p, q in pairs:
            if p != q:
                yield None, (p, q)
                break

    def mapper_compute_rsa(self, _, pair):
        p, q = pair
        n = p * q
        phi = (p - 1) * (q - 1)
        e = 65537

        try:
            d = self.mod_inverse(e, phi)
            yield None, (n, e, d)
        except Exception:
            pass

    def reducer_output_rsa(self, _, rsa_components):
        for n, e, d in rsa_components:
            yield "Modulus n", str(n)
            yield "Public e", str(e)
            yield "Private d", str(d)
```

```
    def miller_rabin(self, n, k):
        if n == 2 or n == 3: return True
        if n % 2 == 0 or n < 2: return False

        r, d = 0, n - 1
        while d % 2 == 0:
            r += 1
            d //= 2

        for _ in range(k):
            a = random.randint(2, n - 2)
            x = pow(a, d, n)

            if x == 1 or x == n - 1:
                continue

            for _ in range(r - 1):
                x = pow(x, 2, n)
                if x == n - 1:
                    break
            else:
                return False
        return True
```

```
    def extended_gcd(self, a, b):
        if a == 0:
            return b, 0, 1
        else:
            g, y, x = self.extended_gcd(b % a, a)
            return g, x - (b // a) * y, y

    def mod_inverse(self, a, m):
        g, x, y = self.extended_gcd(a, m)
        if g != 1:
            raise Exception('Modular inverse does not exist')
        else:
            return x % m

if __name__ == '__main__':
    MRRSAKeyGen.run()
```

```
Writing exercise_5.py
```

Result


0s
Writing exercise_5.py
[18]
0s
!python exercise_5.py exercise_5.txt
No configs found; falling back on auto-configuration
No configs specified for inline runner
Creating temp directory /tmp/exercise_5.root.20251216.080341.126349
Running step 1 of 2...
Running step 2 of 2...
job output is in /tmp/exercise_5.root.20251216.080341.126349/output
Streaming final output from /tmp/exercise_5.root.20251216.080341.126349/output
"Modulus n" "434750289314258717405543354904265737094238255601833837474781
"Public e" "65537"
"Private d" "761743377358687894161748988260934046264514838486267823283390
Removing temp directory /tmp/exercise_5.root.20251216.080341.126349...
exercise_5.txt
1 100